

BANK ACCOUNT MANAGEMENT SYSTEM

Java OOP Program Documentation

1. Question

Write a Java program to create a Bank Account Management System using Object-Oriented Programming concepts.

Create a base Account class containing account number, balance, and account type. Implement operations such as displaying account details, depositing money, withdrawing money, and transferring money between accounts.

Create SavingsAccount and CurrentAccount classes using inheritance. The SavingsAccount should calculate interest, while the CurrentAccount should support an overdraft limit by overriding the withdraw() method.

Create objects for both account types and demonstrate deposit, withdrawal, interest calculation, money transfer, method overriding, inheritance, and constructor chaining using super().

2. Execution Steps

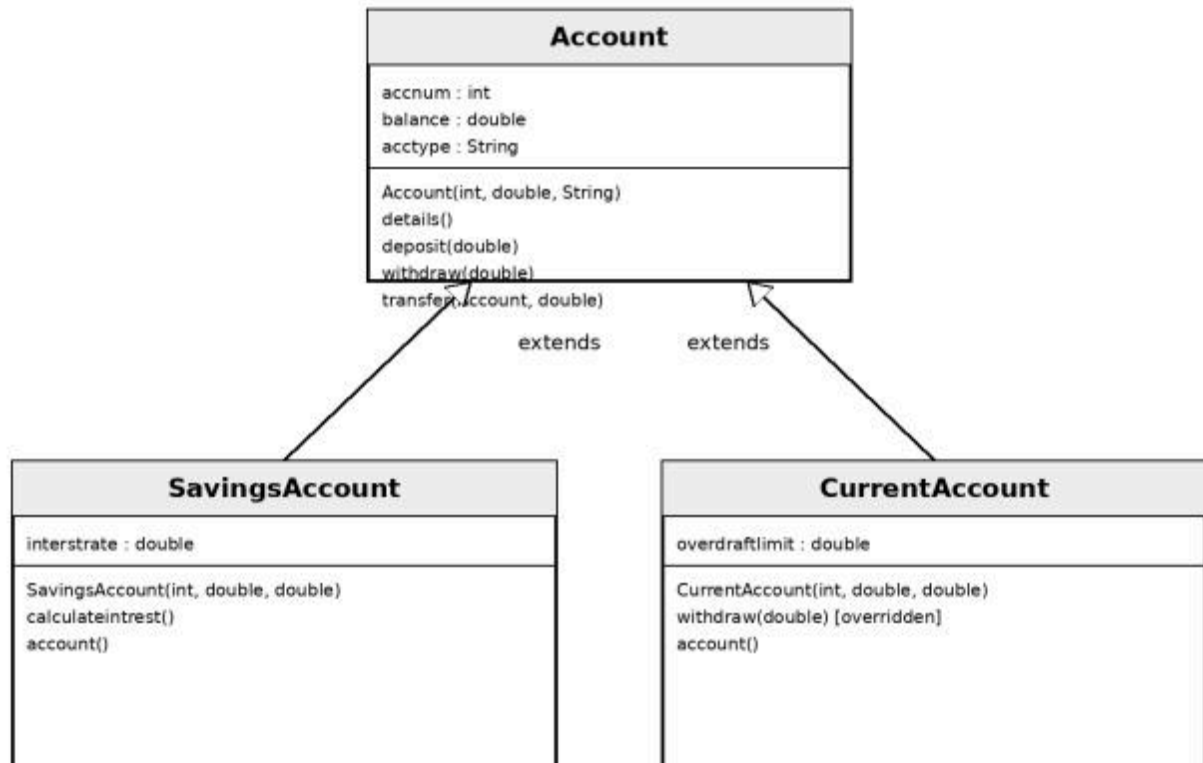
1. Open Eclipse IDE or a terminal.
2. Create a Java project/package named Excercise.
3. Create a Java class named Bank.
4. Copy the complete program into Bank.java.
5. Make sure the public class name is Bank and the file name is Bank.java.
6. Save the program.
7. Compile the program.
8. Run the Bank class containing the main() method.
9. Observe the savings account details.
10. Observe the current account details.
11. Observe deposit, withdrawal, interest, transfer, and final balance operations.
12. Verify the output in the console/terminal.

3. Algorithm

1. Start.
2. Define the Account class with account number, balance, and account type.
3. Create the Account constructor to initialize these values.
4. Define methods for displaying details, depositing, withdrawing, and transferring money.
5. Create SavingsAccount as a subclass of Account.
6. Use super() in its constructor to initialize the Account data.
7. Add an interest rate and calculate interest for the savings account.
8. Create CurrentAccount as a subclass of Account.
9. Add an overdraft limit and override withdraw() so that withdrawal can use the overdraft facility.
10. In main(), create a SavingsAccount object with account number 2000, balance 2424, and interest rate 5%.
11. Create a CurrentAccount object with account number 2154, balance 5845, and overdraft limit 500.
12. Display the initial details of both accounts.
13. Deposit 1000 into the savings account.
14. Withdraw 2000 from the current account.
15. Calculate savings-account interest and add it to the balance.
16. Transfer 500 from the savings account to the current account.
17. Withdraw 5000 from the current account using the overdraft facility.
18. Display the final account details of both SavingsAccount and CurrentAccount.
19. Stop.

4. UML Class Diagram

Account is the base class. SavingsAccount and CurrentAccount inherit from Account, and CurrentAccount overrides withdraw().



5. Java Source Code

```
package Excercise;
class Account{
    int accnum;
    double balance;
    String acctype;

    Account(int n,double m,String l){
        this.accnum=n;
        this.balance=m;
        this.acctype=l;
    }

    void details() {
        System.out.println("accnum :"+accnum);
        System.out.println("balance :"+balance);
        System.out.println("acctype :"+acctype);
    }

    void deposit(double amount) {
        if (amount > 0) {
            balance = balance + amount;
            System.out.println("Deposited: " + amount);
        } else {
            System.out.println("Invalid deposit amount");
        }
    }

    void withdraw(double amount){
        if (amount > 0 && amount <= balance) {
            balance = balance - amount;
            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("Insufficient balance");
        }
    }

    void transfer(Account receiver,double amount) {
        if(amount > 0 && amount <= balance) {
            this.balance = this.balance - amount;
            receiver.balance = receiver.balance + amount;
            System.out.println("Transferred: " + amount);
            System.out.println("To Account: " + receiver.accnum);
        } else {
            System.out.println("Transfer failed");
        }
    }
}

class SavingsAccount extends Account{
    double interstrate;

    SavingsAccount(int n,double m,double k){
        super(n,m,"saving account");
        this.interstrate=k;
    }

    void calculateintrest() {
        double intrest = balance * (interstrate / 100);
        balance = balance + intrest;
        System.out.println("value :"+intrest);
    }

    void account() {
        System.out.println("accnum :"+accnum);
        System.out.println("balance :"+balance);
        System.out.println("interstrate :"+interstrate);
    }
}

class CurrentAccount extends Account{
    double overdraftlimit;

    CurrentAccount(int n,double m,double o){
        super(n,m,"current account");
        this.overdraftlimit=o;
    }

    @Override
    void withdraw(double amount){
        if (amount > 0 && amount <= balance+overdraftlimit) {
            balance = balance - amount;
            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("exceed overdraft limit");
        }
    }

    void account() {
        System.out.println("accnum :"+accnum);
        System.out.println("balance :"+balance);
    }
}
```

```

        System.out.println("overdraftlimit :"+overdraftlimit);
    }
}

public class Bank {
    public static void main(String []args) {
        SavingsAccount s = new SavingsAccount(2000,2424,5);
        CurrentAccount c = new CurrentAccount(2154,5845,500);

        System.out.println("savings account");
        s.account();

        System.out.println("\ncurrent account");
        c.account();

        System.out.println("\nDeposit");
        s.deposit(1000);

        System.out.println("\nwithdraw");
        c.withdraw(2000);

        System.out.println("\ninterstrate");
        s.calculateintrest();

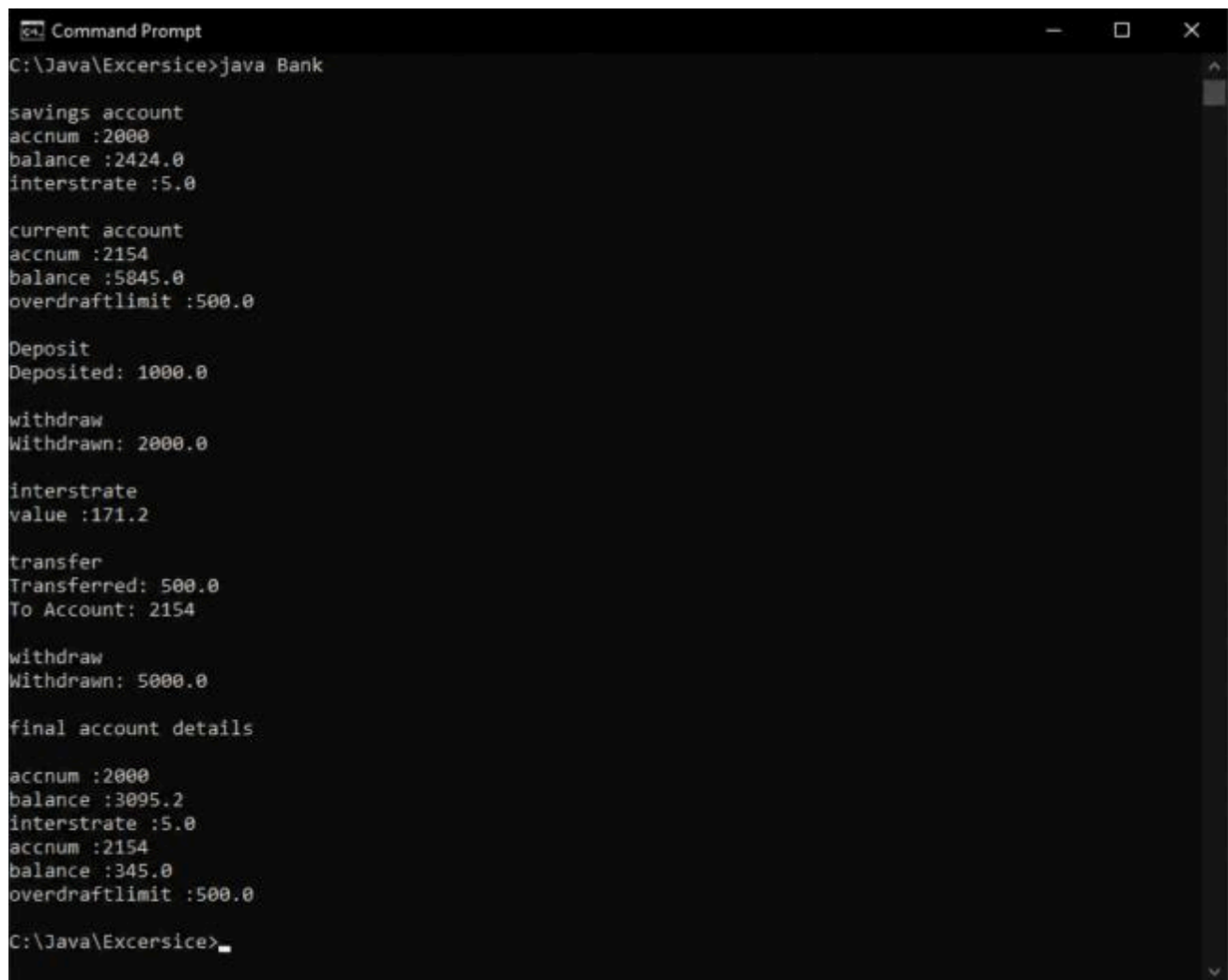
        System.out.println("\ntransfer");
        s.transfer(c,500);

        System.out.println("\nwithdraw");
        c.withdraw(5000);

        System.out.println("\nfinal account details\n");
        s.account();
        c.account();
    }
}

```

6. Output – Terminal Screenshot



```
Command Prompt
C:\Java\Excercise>java Bank

savings account
accnum :2000
balance :2424.0
interstrate :5.0

current account
accnum :2154
balance :5845.0
overdraftlimit :500.0

Deposit
Deposited: 1000.0

withdraw
Withdrawn: 2000.0

interstrate
value :171.2

transfer
Transferred: 500.0
To Account: 2154

withdraw
Withdrawn: 5000.0

final account details

accnum :2000
balance :3095.2
interstrate :5.0
accnum :2154
balance :345.0
overdraftlimit :500.0

C:\Java\Excercise>
```

The screenshot shows a Windows Command Prompt window titled "Command Prompt". The user has executed the command `C:\Java\Excercise>java Bank`. The program output displays the initial state of two bank accounts: a savings account (accnum: 2000, balance: 2424.0, interest rate: 5.0) and a current account (accnum: 2154, balance: 5845.0, overdraft limit: 500.0). It then shows a deposit of 1000.0 into the savings account, a withdrawal of 2000.0 from the savings account, and a withdrawal of 5000.0 from the current account. Finally, it displays the updated account details: savings account balance is 3095.2 and current account balance is 345.0. The prompt ends at `C:\Java\Excercise>`.